Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL604**	**Course Name:**	**Machine Learning Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**04**
Title of the Experiment:	**Implementation of Support Vector Machine Algorithm.**
Date of Performance:	**05/02/2026**
Date of Submission:	**12/02/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Mrs. Sneha Yadav

Signature :

Date:

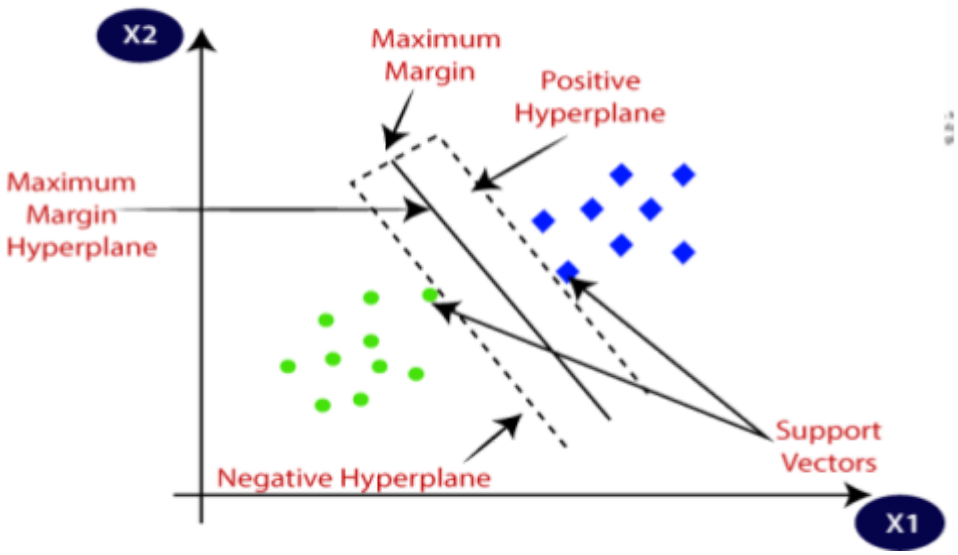
Aim: Implementation of Support Vector Machine Algorithm.

Objective: Ability to perform various feature engineering tasks, apply Support Vector Machine and create the Confusion Matrix.

Theory:

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

Support Vectors:
The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

Implementation of Support Vector Machine

1. Data Pre-processing step
2. Fitting the SVM classifier to the training set
3. Predicting the test set result
4. Creating the confusion matrix
5. Visualizing the training set result
6. Visualizing the test set result

Implementation:

1: Install

```
!pip install scikit-learn matplotlib numpy
```

2: Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

3: Load CSV

```
import pandas as pd

df = pd.read_csv("Pass-Fail Data.csv")
Df
```

	student_id	attendance_pct	homework_pct	midterm_score	study_hours_per_week	pass
0	1	95	92	88	12	1
1	2	88	85	79	10	1
2	3	60	55	58	4	0
3	4	72	70	65	6	1
4	5	40	45	50	3	0
...	...	...	...	...	...	...
95	96	89	92	94	14	1
96	97	79	77	75	8	1
97	98	55	56	58	5	0
98	99	91	90	88	12	1
99	100	52	50	53	4	0

100 rows × 6 columns

```
X = df.drop(columns=["student_id", "pass"])
y = df["pass"]
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
from sklearn.svm import SVC
```

```
svm = SVC(
    kernel="rbf",    # good default
    C=1.0,
    gamma="scale"
)

svm.fit(X_train, y_train)

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
y_pred = svm.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy: 1.0

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00         8
           1       1.00      1.00      1.00        12

    accuracy                           1.00        20
   macro avg       1.00      1.00      1.00        20
weighted avg       1.00      1.00      1.00        20


Confusion Matrix:
 [[ 8  0]
 [ 0 12]]
```

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(scaler.fit_transform(X))
svm_pca = SVC(kernel="rbf", C=1.0, gamma="scale")
svm_pca.fit(X_pca, y)
SVC()
import numpy as np
```

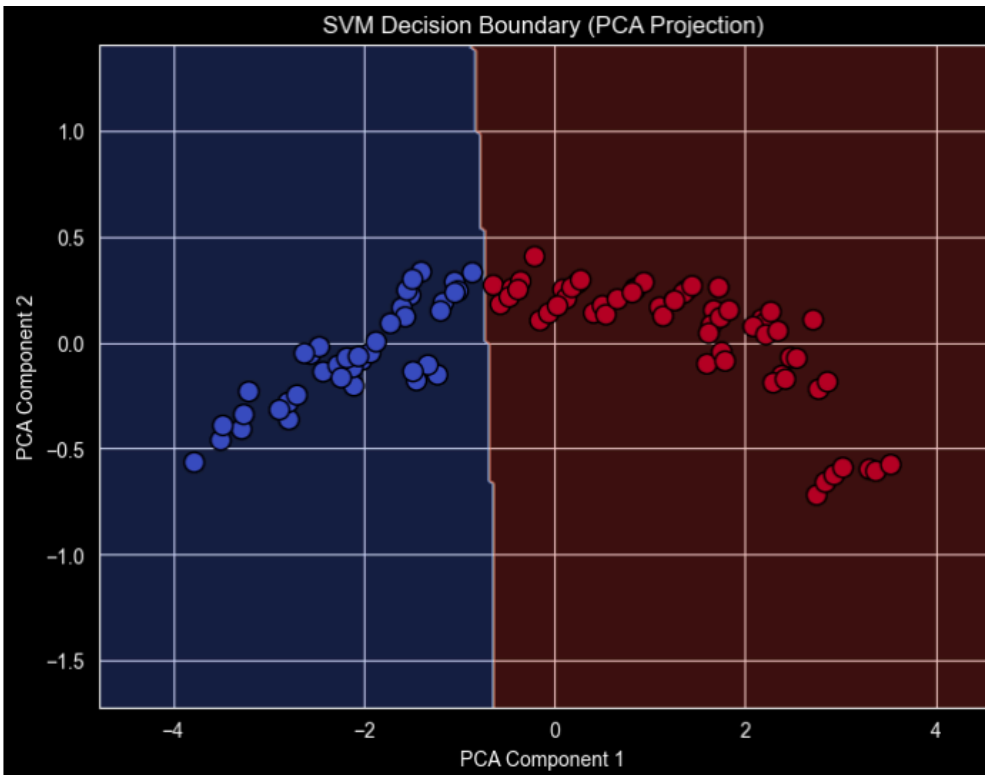
```
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 200),
    np.linspace(y_min, y_max, 200)
)

Z = svm_pca.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap="coolwarm", s=100, edgecolors="k")

plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("SVM Decision Boundary (PCA Projection)")
plt.show()
```



Conclusion:

Support Vector Machines (SVM) are robust supervised learning models designed to find the **optimal hyperplane** that maximizes the margin between different data classes. By focusing on the most critical points, known as **support vectors**, the algorithm ensures the decision boundary is as stable and generalized as possible. Through the use of the **kernel trick**, SVMs can efficiently handle complex, non-linear relationships by projecting data into higher-dimensional spaces for easier classification.